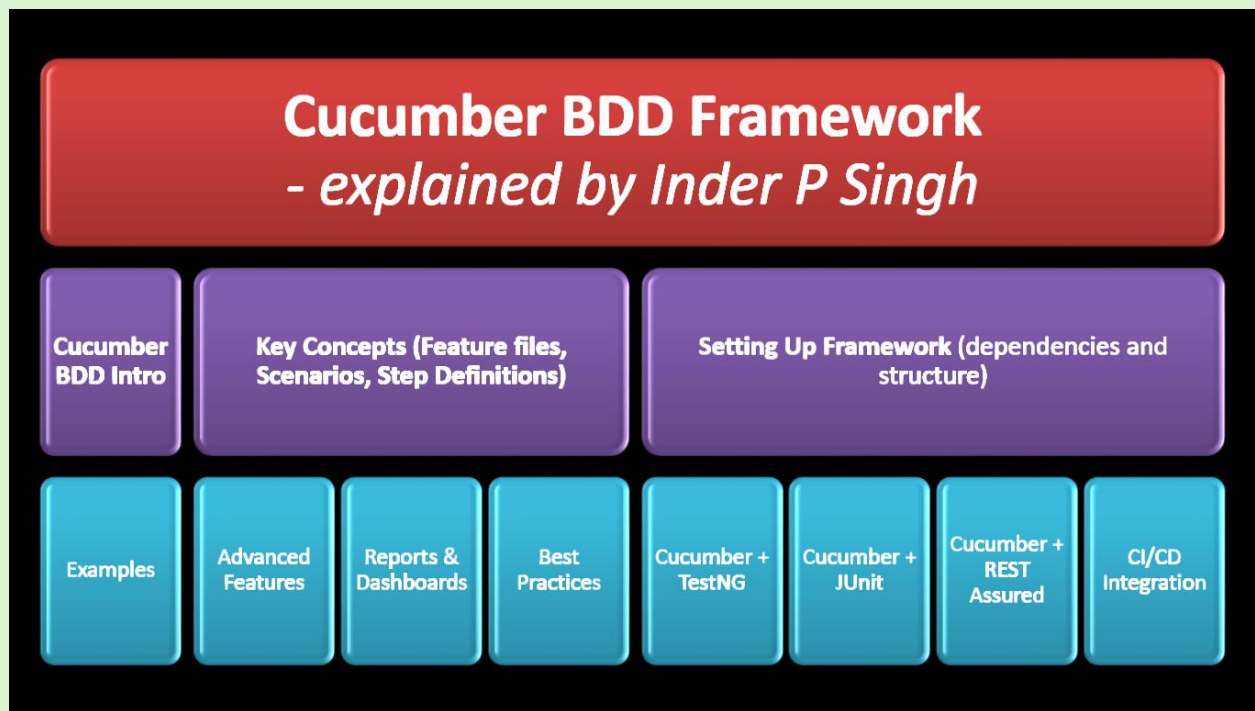


Cucumber BDD

Questions and Answers



Download for reference [↓](#) Like [👍](#) Share [🔗](#)

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

1. Introduction to Cucumber BDD	2
2. Understanding Cucumber BDD Key Concepts	4
3. How to Set Up a Cucumber BDD Framework	7
4. Cucumber BDD with TestNG Framework	11
5. Using Cucumber with JUnit	16
6. Cucumber BDD Framework with Rest Assured for API Testing	19
7. Advanced Cucumber BDD Features	25
8. Cucumber Reporting and Dashboard Setup	28
9. Cucumber BDD Best Practices for Testers	31
10. Cucumber BDD Interview Questions and Answers	34
11. Cucumber BDD Framework Example with Code Snippets	41
12. Handling and Troubleshooting Cucumber BDD Issues	45
13. CI/CD Integration with Cucumber	53

Download for reference  Like  Share 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

1. Introduction to Cucumber BDD

Question: What is Cucumber BDD?

Answer: Cucumber is an open-source tool that supports Behavior-Driven Development, commonly called BDD. It allows teams to write test cases in plain English using a syntax called **Gherkin**. These tests, known as "scenarios" in Cucumber are easy to understand for both technical and non-technical stakeholders. The goal of Cucumber is to help the communication between developers, testers, and business analysts by writing tests according to the business requirements.

For example, a basic Cucumber scenario may look like this:

```
Feature: Login functionality
  Scenario: Valid login
    Given the user is on the login page
    When they enter valid credentials
    Then they should be redirected to the dashboard
```

Benefits of Using Cucumber in Agile Software Development

- **Improved Collaboration:** Cucumber improves collaboration across teams by using a common language syntax (Gherkin) to describe tests. This allows business stakeholders, developers, and testers to all contribute to test scenarios, reducing misunderstanding.
- **Early Testing:** Cucumber works with Agile methodologies, where testing is done early and iteratively. Since the test scenarios are defined before development begins, it helps to build features according to the business requirements from the start.
- **Living Documentation:** Cucumber scenarios are also documentation. These readable test cases describe how the application should behave. They should be constantly updated with each sprint, so that they're up-to-date with the system's functionality.
- **Automation Support:** Cucumber integrates with automation tools and frameworks like Selenium, allowing to automate acceptance tests. As the scenarios are in plain language, testers can link them to code, enabling continuous testing.

Full Form of BDD and Its Importance in the Cucumber Framework

Download for reference  Like  Share 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

BDD stands for **Behavior-Driven Development**. It is an evolution of Test-Driven Development (TDD) where tests are written in the form of expected behaviors of the application. Instead of focusing on technical tests, BDD focuses on the user's perspective. Cucumber helps in implementing BDD because it provides a framework to write these behaviors as tests. The importance of BDD in Cucumber is its focus on collaboration, and guiding the features being developed to be according to the business needs. Cucumber's structure encourages teams to think in terms of "what" the system should do, rather than "how" it should be built. View Cucumber Interview Questions and Answers video in my Software and Testing Training channel or read on.

Download for reference  Like  Share 

Software and Testing Training YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

Software Testing Space Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

2. Understanding Cucumber BDD Key Concepts

Question: What are feature files, step definitions, and Gherkin syntax in Cucumber?

Answer: In Cucumber, these are the elements of how the tests are structured:

Feature Files: These are plain text files written using Gherkin syntax, where scenarios are described. Each feature file represents a specific functionality of the application, such as "User Login" or "Shopping Cart." The example of a feature file is as follows:

```
Feature: User Login
  Scenario: Successful login
    Given the user is on the login page
    When they enter valid credentials
    Then they should be redirected to the dashboard
```

Step Definitions: These are the code implementations of the steps defined in the Gherkin scenarios. Each Gherkin step (Given, When, Then) is linked to a corresponding code block in the step definition file.

Example of a step definition (Java):

```
@Given("the user is on the login page")
public void user_is_on_login_page() {
    // Code to navigate to login page
}
```

Gherkin Syntax: Gherkin is the language used to write feature files. It uses simple language constructs like **Given**, **When**, **Then**, **And**, and **But** to describe the system's behavior in scenarios.

Question: What are the best practices for writing scenarios using Gherkin language?

Download for reference  Like  Share 

[Software and Testing Training](https://www.youtube.com/@QA1SoftwareandTestingTraining) YouTube Channel 81.3K Subscribers [https://youtube.com/@QA1](https://www.youtube.com/@QA1SoftwareandTestingTraining)

[Software Testing Space](https://inderpsingh.blogspot.com/) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

Answer: Writing clear and maintainable Gherkin scenarios is needed to implement Cucumber BDD successfully. Some “best” practices include:

- **Keep it Simple:** Write Gherkin steps in a clear, concise way that non-technical stakeholders can understand. A bad example would be “When the user clicks the button with id 'submitBtn'”. A better example would be “When the user submits the login form”.
- **Single Behavior per Scenario:** Each scenario should test only one specific behavior. Avoid having many steps to a single scenario, which can make it harder to maintain.
- **Avoid Technical Details:** Gherkin scenarios should describe *what* the system should do, not *how* it's implemented. Technical details belong in the step definitions.
- **Use Background, if needed:** If there are repetitive steps that all scenarios share (e.g. logging in), use the **Background** keyword to avoid repeating these steps in every scenario. For example:

```
Background:  
  Given the user is logged in  
  
Scenario: View profile  
  When they visit their profile page  
  Then their profile details should be displayed
```

- **Readable Scenario Names:** Use meaningful names for scenarios so that it's easy to understand their purpose at a glance.

Question: What are Cucumber tags and how are they used in test organization?

Answer: Cucumber tags are used to organize and selectively run scenarios or entire feature files. Tags allow you to group scenarios based on their functionality, priority, or other criteria. Tags are added to the scenario or feature level using @ followed by a name. You can then specify which tags to run when executing tests. On a side note, you should subscribe to Software and Testing Training YouTube channel, if you're interested in learning test automation effectively and quickly. The example of using tags is:

Download for reference  Like  Share 

[Software and Testing Training](https://youtube.com/@QA1) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](https://inderpsingh.blogspot.com/) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

```
@Login @SmokeTest
Scenario: Successful login with valid credentials
  Given the user is on the login page
  When they enter valid credentials
  Then they should see the dashboard
```

You can run tests based on tags, such as only running **@SmokeTest** scenarios. For example, by using the command:

```
mvn test -Dcucumber.options="--tags @SmokeTest"
```

Question: What are the benefits of using Cucumber tags for test organization?

Answer: Cucumber tags provide the following benefits:

- **Selective Test Execution:** Run specific test suites (sets of tests) based on functionality or priority, such as regression tests, smoke tests, or critical paths.
- **Parallel Execution:** Organize your tests to allow parallel execution, for reduced test execution time for large test suites.
- **Categorization:** Tags help in categorizing tests, making it easier to manage and organize them according to feature, sprint, or any other logical classification.

Download for reference  Like  Share 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

3. How to Set Up a Cucumber BDD Framework

Question: How can you integrate Cucumber with Java and build tools like Maven?

Answer: Integrating Cucumber with Java using Maven in test automation has the following steps:

- **Create a Maven Project:** In your IDE (such as IntelliJ or Eclipse), create a new Maven project. This provides a standard way to manage dependencies.
- **Add Cucumber Dependencies in pom.xml:** To use Cucumber, you need to add the required dependencies to your pom.xml file. This typically includes cucumber-java, cucumber-junit, and cucumber-testng (depending on your setup).

Example pom.xml dependencies:

Download for reference  Like  Share 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.


```

<dependencies>
  <!-- Cucumber dependencies -->
  <dependency>
    <groupId>io.cucumber</groupId>
    <artifactId>cucumber-java</artifactId>
    <version>7.0.0</version>
  </dependency>
  <dependency>
    <groupId>io.cucumber</groupId>
    <artifactId>cucumber-junit</artifactId>
    <version>7.0.0</version>
    <scope>test</scope>
  </dependency>
  <!-- JUnit dependency -->
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.13.2</version>
    <scope>test</scope>
  </dependency>
</dependencies>

```

- **Set Up Your Feature Files and Step Definitions:** Create a folder for your feature files and write your test scenarios using Gherkin syntax. Then, create step definition files that link the Gherkin steps to actual Java code.
- **Run Tests with Maven:** To execute the Cucumber tests, you can use Maven commands. The command that will run the tests is `mvn test`

If you're interested in test automation and software testing documents, ensure that you [follow me](https://www.linkedin.com/in/inderpsingh/) in LinkedIn at <https://www.linkedin.com/in/inderpsingh/>

Question: What's an organized folder structure for a Cucumber project?

Download for reference  Like  Share 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

Answer: A structured folder layout helps to maintain a Cucumber BDD project. Here's a suggested folder structure:

```
src
├── test
│   ├── java
│   │   ├── stepDefinitions # Step definition files
│   │   ├── hooks          # Hooks for setup and teardown
│   └── resources
│       └── features        # Gherkin feature files
```

- **features:** Contains all your Gherkin feature files. Each feature is a part of the application's functionality.
- **stepDefinitions:** Java code that implements the Gherkin steps.
- **hooks:** Optional folder where you can add setup and teardown methods (before and after each scenario) using `@Before` and `@After` annotations.

Question: What configuration files are needed to run Cucumber tests?

Answer: The following are the important configuration files for running Cucumber tests:

- **pom.xml (for Maven projects):** As I explained above, `pom.xml` manages all the dependencies required for Cucumber, JUnit, or TestNG. It defines the project and test settings for Maven to execute.
- **cucumber.properties:** This is an optional configuration file for setting environment variables for Cucumber. It can be used to define data like:
 - Default tag expressions
 - Format of the output report
 - Glue paths for step definitions
- **JUnit/TestNG Runner Class:** This file triggers the Cucumber test suite. A runner class is written to specify how to execute the Cucumber tests. If you aren't familiar with JUnit framework, you can view the Software and Testing Training JUnit tutorials at <https://youtu.be/e8Q44fWAXYU> and <https://youtu.be/mZ1Sy5JuFwI>

Download for reference  Like  Share 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

Example of a JUnit Runner class: This class gives Cucumber data on where the feature files are located and how to run them.

```
import org.junit.runner.RunWith;
import io.cucumber.junit.Cucumber;
import io.cucumber.junit.CucumberOptions;

@RunWith(Cucumber.class)
@CucumberOptions(
    features = "src/test/resources/features",
    glue = "stepDefinitions",
    plugin = {"pretty", "html:target/cucumber-reports.html"},
    tags = "@SmokeTest"
)
public class TestRunner {
}
```

- **Log4j or Logback Config File (Optional):** If you want detailed logs for debugging purposes, you can include log configuration files like log4j.properties or logback.xml.

Download for reference  Like  Share 

[Software and Testing Training](https://www.youtube.com/@QA1SoftwareandTestingTraining) YouTube Channel 81.3K Subscribers [https://youtube.com/@QA1](https://www.youtube.com/@QA1SoftwareandTestingTraining)

[Software Testing Space](https://inderpsingh.blogspot.com/) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

4. Cucumber BDD with TestNG Framework

Question: How can you integrate Cucumber with TestNG for test execution?

Answer: Integrating Cucumber with TestNG for test execution is commonly used in Java-based automation frameworks. The steps to do it are:

- **Add Cucumber and TestNG Dependencies:** In your Maven project, add the required dependencies for both Cucumber and TestNG to your pom.xml file. The following is an example of pom.xml dependencies:

```
<dependencies>
  <!-- Cucumber dependencies -->
  <dependency>
    <groupId>io.cucumber</groupId>
    <artifactId>cucumber-java</artifactId>
    <version>7.0.0</version>
  </dependency>
  <dependency>
    <groupId>io.cucumber</groupId>
    <artifactId>cucumber-testng</artifactId>
    <version>7.0.0</version>
  </dependency>

  <!-- TestNG dependency -->
  <dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>7.4.0</version>
    <scope>test</scope>
  </dependency>
</dependencies>
```

- **Create the Runner Class:** Unlike JUnit, TestNG uses a TestNGCucumberRunner class to execute Cucumber tests. A runner class looks something like below. This class acts as a bridge to run Cucumber scenarios with the TestNG testing framework.

Download for reference  Like  Share 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

```
import io.cucumber.testng.AbstractTestNGCucumberTests;
import io.cucumber.testng.CucumberOptions;
import org.testng.annotations.Test;

@CucumberOptions(
    features = "src/test/resources/features",
    glue = "stepDefinitions",
    plugin = {"pretty", "html:target/cucumber-reports.html"}
)
@Test
public class TestNGCucumberRunner extends AbstractTestNGCucumberTests {
}
```

- **Configure testng.xml:** The testng.xml file defines how the tests are executed. It specifies which classes (e.g. your runner class) TestNG will execute. The example testng.xml file is:

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="Cucumber Test Suite">
    <test name="Cucumber Tests">
        <classes>
            <class name="testRunners.TestNGCucumberRunner" />
        </classes>
    </test>
</suite>
```

- **Run the Tests:** Once your runner class and testng.xml are configured, you can run your tests using TestNG either through your IDE or from the command such as:
mvn test -Dsurefire.suiteXmlFiles=testng.xml

Question: What is the role of the testng.xml file and the runner class in Cucumber with TestNG?

Download for reference  Like  Share 

[Software and Testing Training](https://www.youtube.com/@QA1SoftwareandTestingTraining) YouTube Channel 81.3K Subscribers [https://youtube.com/@QA1](https://www.youtube.com/@QA1SoftwareandTestingTraining)

[Software Testing Space](https://inderpsingh.blogspot.com/) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

Answer: The `testng.xml` file and the runner class have roles in executing Cucumber tests within the TestNG framework.

- **testng.xml file:** This configuration file tells TestNG how to organize and execute tests. It specifies which classes to run, allows for grouping, and supports parallel execution of tests. You can also define test suites and control the order of execution, which is useful for managing large test sets.
- **Runner Class:** The runner class launches the Cucumber tests. For TestNG, the runner class extends `AbstractTestNGCucumberTests`, which integrates Cucumber with TestNG. In this class, you define the location of the feature files, step definitions, and any other Cucumber options like reporting plugins.

Question: How do you run Cucumber tests in parallel using TestNG?

Answer: Running Cucumber tests in parallel using TestNG can reduce total execution time, especially when there are many scenarios to test. The steps to set it up are below. If you need code snippets, you can connect to me at <https://www.linkedin.com/in/inderpsingh/>

- **Enable Parallel Execution in testng.xml:** TestNG allows you to run tests in parallel by adding a `parallel` attribute in the `suite` tag of your `testng.xml` file. The example configuration for parallel execution is:

```
<suite name="Cucumber Test Suite" parallel="classes" thread-count="4">
  <test name="Cucumber Tests">
    <classes>
      <class name="testRunners.TestNGCucumberRunner" />
    </classes>
  </test>
</suite>
```

- **Run Scenarios in Parallel:** To run individual Cucumber scenarios in parallel, you need to configure your runner class to enable parallel execution. This can be done by overriding the `scenarios()` method in the runner class. The example of enabling parallel execution in the runner class is below. In this setup, each Cucumber scenario

Download for reference  Like  Share 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

will run in parallel, using the `parallel = true` attribute in the `@DataProvider` annotation.

```
@CucumberOptions(  
    features = "src/test/resources/features",  
    glue = "stepDefinitions",  
    plugin = {"pretty", "html:target/cucumber-reports.html"}  
)  
public class TestNGCucumberRunner extends AbstractTestNGCucumberTests {  
  
    @DataProvider(parallel = true)  
    @Override  
    public Object[][] scenarios() {  
        return super.scenarios();  
    }  
}
```

- **Run the Tests:** You can then execute the tests using Maven (see example below) or from your IDE. TestNG will run the tests in parallel across multiple threads as defined.
`mvn test -Dsurefire.suiteXmlFiles=testng.xml`

Download for reference  Like  Share 

[Software and Testing Training](https://www.youtube.com/@QA1SoftwareandTestingTraining) YouTube Channel 81.3K Subscribers [https://youtube.com/@QA1](https://www.youtube.com/@QA1SoftwareandTestingTraining)

[Software Testing Space](https://inderpsingh.blogspot.com/) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

5. Using Cucumber with JUnit

Question: How do you set up Cucumber with JUnit for running BDD tests?

Answer: Setting up Cucumber with JUnit for running BDD tests is common in Java-based automation frameworks. The steps to get it working:

- **Add Dependencies:** In your Maven project's `pom.xml` file, add the Cucumber and JUnit dependencies (shown above).
- **Create a JUnit Runner Class:** JUnit uses the `@RunWith` annotation (shown above) to execute Cucumber tests. The runner class defines where your feature files and step definitions are located.
- **Write Feature Files:** Cucumber feature files (with `.feature` file extension) contain your BDD scenarios written in Gherkin syntax. For example:

```
Feature: Login functionality
  Scenario: User logs in successfully
    Given the user is on the login page
    When the user enters valid credentials
    Then the user should be redirected to the homepage
```

- **Create Step Definitions:** The step definitions implement the logic for each step in the feature file. The example step definition for the scenario is:

Download for reference  Like  Share 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.


```

import io.cucumber.java.en.Given;
import io.cucumber.java.en.When;
import io.cucumber.java.en.Then;

public class LoginSteps {
    @Given("the user is on the login page")
    public void userOnLoginPage() {
        // Code to navigate to login page
    }

    @When("the user enters valid credentials")
    public void userEntersCredentials() {
        // Code to enter credentials
    }

    @Then("the user should be redirected to the homepage")
    public void redirectToHomepage() {
        // Code to verify user is on homepage
    }
}

```

- **Run the Tests:** You can run the JUnit runner class either from your IDE or via Maven using the command like `mvn test`

You can see more examples in my [Cucumber Interview Questions and Answers](https://youtu.be/dvhd5F0ckSE) video at <https://youtu.be/dvhd5F0ckSE>

Question: What are the differences between TestNG and JUnit for running Cucumber tests?

Answer: TestNG and JUnit frameworks used to execute Cucumber tests have some differences in terms of features and flexibility:

Difference	JUnit	TestNG
------------	-------	--------

Download for reference [↓](#) Like [👍](#) Share [🔗](#)

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

Annotations	Uses simple annotations like <code>@Before</code> , <code>@After</code> , and <code>@Test</code> .	Has more advanced annotations like <code>@BeforeClass</code> , <code>@AfterClass</code> , <code>@BeforeSuite</code> , and <code>@AfterSuite</code> , which allow for more levels of control over test execution.
Test Configuration	No built-in configuration file; all configuration is done in code, typically in the runner class or via custom setups.	Uses <code>testng.xml</code> for advanced configuration, which can be used to define test suites, groups, and enable parallel execution.
Parallel Execution	Running tests in parallel requires extra setup or third-party plugins like Maven Surefire. JUnit 4 doesn't have built-in support for parallel execution.	Has built-in support for parallel execution with a simple configuration in the <code>testng.xml</code> file.
Flexibility	Lightweight and suitable for simple test cases but lacks the advanced features of TestNG.	More powerful in terms of grouping tests, running them in parallel, and generating advanced reports.
Dependency Injection	Supports dependency injection with frameworks like Spring, but configuration can be more complex.	Built-in dependency injection, making it more flexible for test setups requiring external dependencies.

Download for reference  Like  Share 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

6. Cucumber BDD Framework with Rest Assured for API Testing

Question: How can you integrate Rest Assured with Cucumber for API testing?

A: To integrate Rest Assured with Cucumber for API testing, follow these steps:

- **Add Dependencies:** Add Cucumber and Rest Assured dependencies to your `pom.xml` if you're using Maven.

```
<dependencies>
  <!-- Cucumber Dependencies -->
  <dependency>
    <groupId>io.cucumber</groupId>
    <artifactId>cucumber-java</artifactId>
    <version>7.0.0</version>
  </dependency>
  <dependency>
    <groupId>io.cucumber</groupId>
    <artifactId>cucumber-junit</artifactId>
    <version>7.0.0</version>
  </dependency>

  <!-- Rest Assured Dependencies -->
  <dependency>
    <groupId>io.rest-assured</groupId>
    <artifactId>rest-assured</artifactId>
    <version>4.4.0</version>
  </dependency>
</dependencies>
```

- **Create Feature File:** Create a Cucumber feature file for your API test cases using Gherkin syntax. This describes the behavior of the API. Example feature file:

Download for reference  Like  Share 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

Feature: API testing with Rest Assured

Scenario: Verify API status and response for user details

Given I set the API endpoint "/users/1"

When I send a GET request to the API

Then I should receive a 200 status code

And the response body should contain the user name "John Doe"

- **Write Step Definitions:** The step definitions use Rest Assured to handle API requests and responses. You should implement the logic here for the steps defined in the feature file. Example step definition using Rest Assured:

Download for reference  Like  Share 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

```

import io.cucumber.java.en.Given;
import io.cucumber.java.en.When;
import io.cucumber.java.en.Then;
import static io.restassured.RestAssured.*;
import io.restassured.response.Response;
import org.junit.Assert;

public class ApiStepDefinitions {

    private String apiEndpoint;
    private Response response;

    @Given("I set the API endpoint {string}")
    public void setApiEndpoint(String endpoint) {
        apiEndpoint = "https://jsonplaceholder.typicode.com" + endpoint;
    }

    @When("I send a GET request to the API")
    public void sendGetRequest() {
        response = given().when().get(apiEndpoint);
    }

    @Then("I should receive a {int} status code")
    public void verifyStatusCode(int statusCode) {
        Assert.assertEquals(statusCode, response.getStatusCode());
    }

    @Then("the response body should contain the user name {string}")
    public void verifyResponseBody(String userName) {
        String name = response.jsonPath().getString("name");
        Assert.assertEquals(userName, name);
    }
}

```

Download for reference  Like  Share 

Software and Testing Training YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

Software Testing Space Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

- **Run the Tests:** Finally, you run the tests via your Cucumber runner class or with Maven. The Cucumber runner class executes the feature file and maps the steps to the Rest Assured API interactions. Example JUnit runner:

```
import org.junit.runner.RunWith;
import io.cucumber.junit.Cucumber;
import io.cucumber.junit.CucumberOptions;

@RunWith(Cucumber.class)
@CucumberOptions(
    features = "src/test/resources/features",
    glue = "stepDefinitions",
    plugin = {"pretty", "html:target/cucumber-reports.html"}
)
public class ApiTestRunner {
}
```

Question: How can you write API scenarios using Gherkin in Cucumber?

Answer: Writing API scenarios in Gherkin for Cucumber makes them human-readable. Here's how you can write API scenarios:

- **Define the Feature:** The first line of the feature file starts with **Feature :** and provides a high-level description of what you're testing. For API testing, it could be about validating the functionality of a particular endpoint. Example:
Feature: API testing for User details endpoint
- **Define the Scenario:** Each scenario describes a specific test case. It begins with the **Scenario:** keyword followed by a descriptive title. For API testing, this might include checking the status code, headers, or response body. Example:
Scenario: Verify API response for user details
- **Write Gherkin Steps:** Use **Given**, **When**, and **Then** steps to define the conditions, actions, and expected outcomes of the API call.
 - **Given** sets up the initial context, such as the API endpoint.

Download for reference  Like  Share 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

- When describes the action, such as sending a request.
- Then checks the expected result, such as the status code or response content.

Example:

```
Given I set the API endpoint "/users/1"
When I send a GET request to the API
Then I should receive a 200 status code
And the response body should contain the user name "John Doe"
```

Question: How can you manage JSON responses and API requests in Cucumber BDD?

Answer: Managing JSON responses and API requests in Cucumber BDD is done using the step definitions where you interact with the APIs using Rest Assured. Here's how you can handle API requests and JSON responses:

- **Sending API Requests:** You can use Rest Assured to send various types of HTTP requests (GET, POST, PUT, DELETE). For example, you can send a GET request as `response = given().when().get(apiEndpoint);`

For a POST request with a JSON payload:

```
String jsonBody = "{ \"name\": \"John Doe\", \"email\": \"john@example.com\" }";
response = given().header("Content-Type", "application/json")
    .body(jsonBody)
    .when()
    .post(apiEndpoint);
```

- **Extracting Data from JSON Responses:** Rest Assured has the feature to parse JSON responses. You can extract values from the response body using methods like `jsonPath()`. Example of extracting a value:

```
String userName = response.jsonPath().getString("name");
```

- **Assertions on JSON Responses:** Once you've extracted the necessary data from the JSON response, you can assert that it matches the expected result. Example:

Download for reference  Like  Share 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

```
Assert.assertEquals("John Doe", userName);
```

- **Handling Complex JSON:** If the API returns a complex JSON response with nested objects, you can use `jsonPath` to navigate the structure. Example of accessing nested JSON data:

```
String city = response.jsonPath().getString("address.city");
```

Download for reference  Like  Share 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

7. Advanced Cucumber BDD Features

Question: How can you implement data-driven testing in Cucumber using Scenario Outline and Examples?

Answer: Data-driven testing in Cucumber is achieved using the Scenario Outline and Examples keywords. Scenario Outline allows you to write a single scenario and run it multiple times with different sets of test data. The Examples keyword is used to define the data. In the example below, the same steps are executed for each row of data in the Examples table. Cucumber will replace the placeholders (<username>, <password>, <message>) with the corresponding values from each row during execution.

```
Scenario Outline: Test login functionality with multiple users
  Given the user navigates to the login page
  When the user enters username "<username>" and password "<password>"
  Then the user should see the message "<message>"
```

Examples:

username	password	message
user1	pass1	Login successful
user2	pass2	Login successful
invalid	invalid	Login failed

Question: What are Hooks in Cucumber and how are they used?

Answer: Hooks in Cucumber allow you to run blocks of code before or after each scenario. They set up preconditions and clean up after test execution. These hooks are useful where you need to initialize resources like databases or browsers before the tests run and release resources or reset states afterward. There are two main types of hooks:

- @Before: This hook runs before each scenario.
- @After: This hook runs after each scenario.

For example:

Download for reference  Like  Share 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

```
import io.cucumber.java.Before;
import io.cucumber.java.After;

public class Hooks {

    @Before
    public void setUp() {
        System.out.println("This will run before the Scenario");
        // Code to set up test data or browser initialization
    }

    @After
    public void tearDown() {
        System.out.println("This will run after the Scenario");
        // Code to clean up after scenario execution
    }
}
```

Question: What is Cucumber Glue and how does it link step definitions to scenarios?

Answer: In Cucumber, the term **glue** refers to the code that connects the Gherkin steps in feature files to their corresponding step definitions in Java (or any other language that Cucumber supports). The `glue` option in the Cucumber runner class defines the package where the step definition files are located.

For example, if your step definitions are in the `stepDefinitions` package, you would specify the glue path in the runner class like below. The `glue` links the feature steps to the step definition methods that contain the logic for each step. Without the correct glue path, Cucumber will not know where to find the step definitions, and the tests will fail to execute properly.

Download for reference  Like  Share 

Software and Testing Training YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

Software Testing Space Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

```
import io.cucumber.junit.Cucumber;
import org.junit.runner.RunWith;
import io.cucumber.junit.CucumberOptions;

@RunWith(Cucumber.class)
@CucumberOptions(
    features = "src/test/resources/features",
    glue = "stepDefinitions", // This is where glue is defined
    plugin = {"pretty", "html:target/cucumber-reports"}
)
public class TestRunner {
}
```

Download for reference  Like  Share 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

8. Cucumber Reporting and Dashboard Setup

Question: How can you generate and view HTML and JSON reports from Cucumber test runs?

Answer: Cucumber can generate several types of reports after test execution, including HTML and JSON reports. To generate these reports, you need to configure the `CucumberOptions` in your test runner class by specifying the report plugins.

Below's an example of how to generate HTML and JSON reports.

- The `html:target/cucumber-reports/Cucumber.html` option generates an HTML report that can be viewed in any browser. After the test run, navigate to the `target/cucumber-reports` folder and open the `Cucumber.html` file to view the results.
- The `json:target/cucumber-reports/Cucumber.json` option generates a JSON report, which is helpful if you want to integrate with other reporting tools or dashboards that can parse JSON data.
- These reports provide a detailed breakdown of passed, failed, and skipped scenarios, along with the steps and the execution time for each.

Download for reference  Like  Share 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

```

import io.cucumber.junit.Cucumber;
import io.cucumber.junit.CucumberOptions;
import org.junit.runner.RunWith;

@RunWith(Cucumber.class)
@CucumberOptions(
    features = "src/test/resources/features",
    glue = "stepDefinitions",
    plugin = {
        "pretty", // Console output
        "html:target/cucumber-reports/Cucumber.html", // HTML report
        "json:target/cucumber-reports/Cucumber.json" // JSON report
    }
)
public class TestRunner {
}

```

Question: How can you integrate Cucumber reporting with Jenkins for continuous integration?

Answer: Integrating Cucumber reports with Jenkins is a practice in continuous integration (CI). Jenkins can display Cucumber HTML and JSON reports and visualize trends over time. By integrating Cucumber with Jenkins, teams can automate their test reporting and maintain a history of test execution results for each build, helping in identifying trends and issues over time. To set this up:

- **Install the Cucumber Reports Plugin:** Go to Jenkins dashboard. Navigate to Manage Jenkins > Manage Plugins > Available tab. Search for **Cucumber Reports Plugin** and install it.
- **Configure Jenkins Job to Execute Cucumber Tests:** In your Jenkins job, ensure that your build tool (e.g. Maven or Gradle) is running the Cucumber tests. For Maven, you might add the following command in the Build step: `mvn clean test`
- **Post-Build Configuration for Cucumber Reports:** Once the tests run, configure Jenkins to publish the reports. Under Post-build Actions, click Add post-build action

Download for reference  Like  Share 

[Software and Testing Training](https://www.youtube.com/@QA1) YouTube Channel 81.3K Subscribers [https://youtube.com/@QA1](https://www.youtube.com/@QA1)

[Software Testing Space](https://inderpsingh.blogspot.com/) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

and select **Publish Cucumber Reports**. Provide the path to the JSON report generated by Cucumber (e.g. `target/cucumber-reports/Cucumber.json`).

- **View Cucumber Reports in Jenkins:** After the job is executed, you'll see the Cucumber reports linked in the Jenkins job. The report shows detailed test results, scenario statuses, and trends over multiple test runs. Example JSON path configuration in Jenkins is: `target/cucumber-reports/Cucumber.json`

Download for reference  Like  Share 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

9. Cucumber BDD Best Practices for Testers

Question: How can you write reusable and maintainable step definitions in Cucumber?

Answer: Writing reusable and maintainable step definitions in Cucumber reduces code duplication. Here are some “best” practices:

- **Use Parameterization:** Instead of writing multiple step definitions for different inputs, use placeholders (parameters) in your Gherkin scenarios. This allows one step definition to handle multiple variations of a step. For example, consider the step: Given I login with username "admin" and password "admin123"
The Step definition could be:

```
@Given("^I login with username \"([^\"]*)\" and password \"([^\"]*)\"$")
public void login(String username, String password) {
    // Code to log in with username and password
}
```

- **Group Similar Steps:** If multiple steps share similar logic, refactor them into helper methods within the step definition class. Alternately, create reusable utilities to keep your step definitions clean and focused on behavior.
- **Avoid Hardcoding Data:** Use external data sources, such as configuration files or data tables in Gherkin, to keep the step definitions flexible.
- **Use Consistent Naming Conventions:** Stick to a consistent format for your method names in step definitions so that they are easy to identify and understand.

Question: How can large feature files be refactored for better scalability and readability?

Answer: Refactoring large feature files improves their scalability and readability. Here are some techniques:

- **Modularize Feature Files:** Split large feature files into smaller, more focused ones based on functionality. For example, instead of having one feature file for all login scenarios, create separate files like login.feature, reset-password.feature, etc. This keeps tests focused and easier to maintain.

Download for reference  Like  Share 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

- **Use Background Keyword:** If there are common steps at the beginning of several scenarios, you can place those steps under the Background section. This prevents repetitive steps and keeps the scenarios clean. View the Background example above. By the way, if you find such documents useful, you can follow me in LinkedIn at <https://www.linkedin.com/in/inderpsingh/>
- **Scenario Outlines:** Use Scenario Outline and Examples to group similar scenarios into one. This reduces redundancy in the feature file. View the Scenario Outline example above.
- **Comment and Organize:** Comment sections of the feature file to explain complex scenarios or blocks of steps. Also, group related scenarios together logically.

Question: How can you avoid duplication in Cucumber step definitions?

Answer: Avoiding duplication in step definitions keeps the test suite maintainable and efficient. Here's how you can avoid duplication:

- **Use Regular Expressions:** Define your steps with regular expressions that can match a variety of input patterns. This allows one step definition to cover multiple steps in Gherkin. For example, instead of writing separate step definitions for each possible assertion, use a regex that handles different outcomes. Java example:

```
@Then("^I should see the message \"([^\"]*)\"$")
public void verifyMessage(String message) {
    // Code to verify the message
}
```

- **Reuse Common Logic:** If two or more step definitions share the same logic, move that logic to a helper function or utility class. The step definition can then call the shared method, reducing code duplication.
- **Step Definition Sharing Across Files:** It should be possible to reuse step definitions across feature files. Don't repeat step definitions in multiple classes unless necessary. Group related step definitions logically.
- **Avoid Ambiguity:** Make sure your step definitions don't accidentally match other steps. This can lead to unwanted step reuse. Use specific wording and matchers in your regular expressions to avoid conflicts.

Download for reference  Like  Share 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

Download for reference  **Like**  **Share** 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

10. Cucumber BDD Interview Questions and Answers

Question: What is Cucumber BDD? Why is it used in software development?

Answer: Cucumber BDD (Behavior-Driven Development) is a framework that allows developers and testers to write test cases in plain, understandable language. It uses Gherkin syntax, which bridges the gap between technical and non-technical stakeholders by making test scenarios readable to everyone. The main goal of BDD and Cucumber is to improve communication between business analysts, developers, and testers in order to create the software that matches to the business requirements. It is particularly useful in Agile environments for continuous collaboration and test automation.

Question: What is the full form of BDD, and why is it important in the Cucumber framework?

Answer: The full form of BDD is **Behavior-Driven Development**. BDD is important in Cucumber because it focuses on the behavior of an application from a user's perspective rather than its implementation. This helps the team focus on delivering the features that matter the most to users. Cucumber implements BDD by allowing test scenarios to be written in plain English (using Gherkin), which refines the collaboration between developers, testers, and business stakeholders.

Question: What are feature files in Cucumber, and what role do they play in BDD?

Answer: Feature files in Cucumber define the test scenarios for a particular functionality of an application. Each feature file consists of one or more scenarios written using Gherkin syntax. The scenarios describe a specific behavior of the system and follow a **Given-When-Then** structure. Feature files are essential in BDD because they document how the system is expected to behave in the way that non-technical team members can understand. For more examples, you can view my Cucumber Interview Questions and Answers video at <https://youtu.be/dvhd5F0ckSE> but one example of a feature file is as follows:

Download for reference  Like  Share 

[Software and Testing Training](https://www.youtube.com/@QA1SoftwareandTestingTraining) YouTube Channel 81.3K Subscribers [https://youtube.com/@QA1](https://www.youtube.com/@QA1SoftwareandTestingTraining)

[Software Testing Space](https://inderpsingh.blogspot.com/) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

Feature: Login functionality

Scenario: Successful login

Given the user is on the login page

When the user enters valid credentials

Then the user should be redirected to the dashboard

Question: How do you define step definitions in Cucumber, and what is their purpose?

Answer: Step definitions in Cucumber link the plain text steps written in feature files to the code that performs the actual actions described. Each step in the feature file corresponds to a method in the step definition class, written in a programming language like Java, which contains the logic to execute the step. For example,

```
@Given("^the user is on the login page$")
public void navigateToLoginPage() {
    // Code to navigate to the login page
}

@When("^the user enters valid credentials$")
public void enterCredentials() {
    // Code to input credentials
}

@Then("^the user should be redirected to the dashboard$")
public void verifyDashboard() {
    // Code to verify the dashboard page
}
```

Question: What is the difference between TestNG and JUnit when integrating with Cucumber?

Answer: Both TestNG and JUnit are testing frameworks used to run Cucumber tests, but they have differences:

Download for reference  Like  Share 

[Software and Testing Training](https://youtube.com/@QA1) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](https://inderpsingh.blogspot.com/) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

- **TestNG** offers more advanced features like parallel test execution, better support for annotations, and dependency management between test methods.
- **JUnit** is simpler and commonly used with Cucumber for its lightweight nature and faster integration.

In a Cucumber setup:

- With **TestNG**, you use the `@Test` annotation and configure tests in the `testng.xml` file.
- With **JUnit**, you run tests via the `@RunWith(Cucumber.class)` annotation, and tests are executed directly.

Question: How can you integrate Cucumber with Rest Assured for API testing?

Answer: To integrate Cucumber with Rest Assured for API testing, you follow these steps:

- **Add Dependencies:** Include Cucumber, Rest Assured, and TestNG or JUnit dependencies in your `pom.xml` file (if using Maven).
- **Write API Scenarios:** Define API test scenarios in Gherkin within the feature files. For example, a scenario for validating a GET API call.
- **Step Definitions:** Write step definitions using Rest Assured methods like `given()`, `when()`, and `then()` to test the API.

Example:

```
@When("^I send a GET request to the endpoint$")
public void sendGetRequest() {
    response = given().when().get("/api/v1/users");
}

@Then("^I should receive status code 200$")
public void validateStatusCode() {
    response.then().statusCode(200);
}
```

Download for reference  Like  Share 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

- **Run Tests:** Execute the Cucumber tests using JUnit or TestNG, and verify the API responses using assertions provided by Rest Assured.

Question: What are Hooks in Cucumber, and how do they enhance test execution?

Answer: Hooks in Cucumber are special blocks of code that run before or after each scenario or feature. They are annotated with `@Before` and `@After`. Hooks are typically used for setup and teardown activities, such as launching the browser or initializing the database before a scenario, and closing the browser or cleaning up data after a scenario completes.

Note: Please give me feedback on my document in LinkedIn by messaging me at <https://www.linkedin.com/in/inderpsingh/>

Hooks

example:

```
import io.cucumber.java.Before;
import io.cucumber.java.After;

public class Hooks {

    @Before
    public void setUp() {
        System.out.println("This will run before the Scenario");
        // Code to set up test data or browser initialization
    }

    @After
    public void tearDown() {
        System.out.println("This will run after the Scenario");
        // Code to clean up after scenario execution
    }
}
```

Question: How does Cucumber support parallel execution with TestNG?

Download for reference  Like  Share 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

Answer: Cucumber can run scenarios in parallel when integrated with TestNG. To achieve this, you configure the `testng.xml` file and set the `parallel` attribute to either `methods`, `classes`, or `tests`, depending on how you want to parallelize the execution. Additionally, Cucumber provides support for parallel execution through tools like the `cucumber-jvm-parallel-plugin`. Parallel execution improves test speed, especially when testing large suites, by running multiple tests at the same time. The example `testng.xml` configuration for parallel execution is below:

```
<suite name="ParallelSuite" parallel="tests" thread-count="4">
  <test name="Test1">
    <classes>
      <class name="com.example.tests.TestRunner" />
    </classes>
  </test>
</suite>
```

Question: What are some advanced features of Cucumber?

Answer: Advanced features in Cucumber that I know include:

- **Data-Driven Testing with Scenario Outline:** This allows running the same test scenario with different data sets by using placeholders and the Examples table. The example of data-driven testing with Scenario Outline is

```
Scenario Outline: Test login functionality with multiple users
  Given I login with username "<username>" and password "<password>"
  Then I should see the dashboard
```

Examples:

username	password
admin	admin123
user1	userpass1

- **Tags:** Cucumber tags allow you to organize and selectively execute specific scenarios or feature files. You can run tests tagged as `@Smoke` or `@Regression`, depending on the need.

Download for reference  Like  Share 

[Software and Testing Training](https://www.youtube.com/@QA1SoftwareandTestingTraining) YouTube Channel 81.3K Subscribers [https://youtube.com/@QA1](https://www.youtube.com/@QA1SoftwareandTestingTraining)

[Software Testing Space](https://inderpsingh.blogspot.com/) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

- **Cucumber Glue:** Glue refers to the package or directory where step definitions are located. This helps Cucumber link the feature file steps with the corresponding Java methods.

Question: How can you integrate Cucumber with Jenkins for CI/CD?

Answer: Integrating Cucumber with Jenkins enables continuous integration (CI) and continuous deployment (CD). This setup allows automated execution and reporting of Cucumber tests, ensuring continuous feedback during development. The steps include:

- **Create a Jenkins Job:** Set up a Jenkins job and configure the source code repository (e.g. Git).
- **Add Build Steps:** Add build steps to run Cucumber tests via Maven or Gradle.
- **Post-Build Actions:** Use plugins like the **Cucumber Reports** plugin to generate and visualize Cucumber test results as HTML or JSON reports.
- **Scheduling Jobs:** Schedule the Jenkins job to run automatically after every code push or periodically.

Download for reference  Like  Share 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

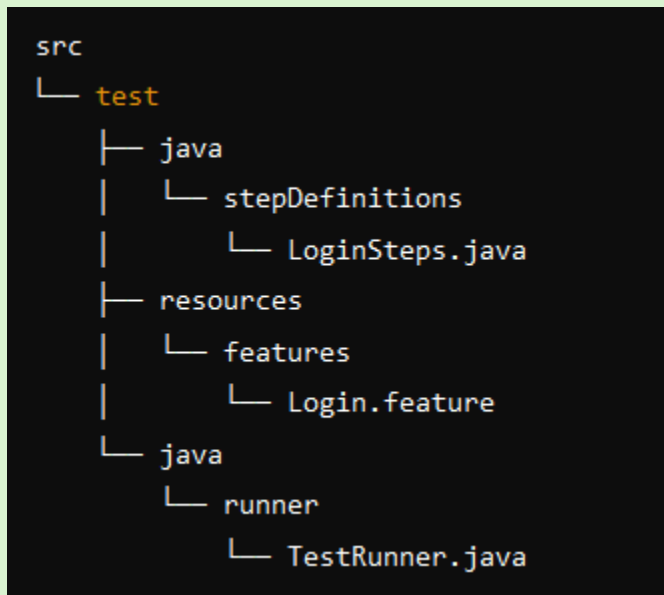
Copyright © 2025 All Rights Reserved.

11. Cucumber BDD Framework Example with Code Snippets

Question: Can you provide a full working example of a Cucumber BDD project?

Answer: Sure! The following is an example of a Cucumber BDD framework for a login functionality:

- **Folder Structure:**



- **Maven Dependencies (pom.xml):** The pom.xml file should have the required dependencies for Cucumber, JUnit, and Selenium (or any other web driver framework you use).

Download for reference [📄](#) Like [👍](#) Share [🔗](#)

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.


```
<dependencies>
  <!-- Cucumber Java -->
  <dependency>
    <groupId>io.cucumber</groupId>
    <artifactId>cucumber-java</artifactId>
    <version>7.0.0</version>
  </dependency>

  <!-- Cucumber JUnit -->
  <dependency>
    <groupId>io.cucumber</groupId>
    <artifactId>cucumber-junit</artifactId>
    <version>7.0.0</version>
    <scope>test</scope>
  </dependency>

  <!-- Selenium -->
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>4.0.0</version>
  </dependency>
</dependencies>
```

Question: Can you give an example feature file for Cucumber?

Answer: Feature files are written in Gherkin syntax. Each scenario describes a specific behavior that needs to be tested. Here's an example of a feature file for a login functionality.

Download for reference  Like  Share 

[Software and Testing Training](https://www.youtube.com/@QA1SoftwareandTestingTraining) YouTube Channel 81.3K Subscribers [https://youtube.com/@QA1](https://www.youtube.com/@QA1SoftwareandTestingTraining)

[Software Testing Space](https://inderpsingh.blogspot.com/) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

Feature: Login functionality

Scenario: Successful login with valid credentials

Given the user is on the login page

When the user enters valid credentials

And clicks the login button

Then the user should be redirected to the dashboard

Scenario: Login attempt with invalid credentials

Given the user is on the login page

When the user enters invalid credentials

And clicks the login button

Then an error message should be displayed

Question: How are step definitions written in Cucumber?

Answer: Step definitions are Java methods that execute the steps mentioned in the feature files. They are annotated with the Gherkin steps (like @Given, @When, @Then) and contain the code to interact with the application. The example of **LoginSteps.java** is below. In this example, the step definitions map the steps from the feature file to the actual automation logic that interacts with the login page.

Download for reference  Like  Share 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

```

package stepDefinitions;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import io.cucumber.java.en.*;

public class LoginSteps {

    WebDriver driver;

    @Given("the user is on the login page")
    public void the_user_is_on_the_login_page() {
        // Initialize WebDriver and navigate to login page
        System.setProperty("webdriver.chrome.driver", "path/to/chromedriver");
        driver = new ChromeDriver();
        driver.get("https://example.com/login");
    }

    @When("the user enters valid credentials")
    public void the_user_enters_valid_credentials() {
        // Enter valid credentials
        driver.findElement(By.id("username")).sendKeys("testuser");
        driver.findElement(By.id("password")).sendKeys("testpassword");
    }

    @When("clicks the login button")
    public void clicks_the_login_button() {
        // Click the login button
        driver.findElement(By.id("loginbutton")).click();
    }

    @Then("the user should be redirected to the dashboard")
    public void the_user_should_be_redirected_to_the_dashboard() {
        // Validate successful login
        String currenturl = driver.getCurrentUrl();
        assert currenturl.equals("https://example.com/dashboard");
        driver.quit();
    }

    @Then("an error message should be displayed")
    public void an_error_message_should_be_displayed() {
        // Validate error message
        String errorMsg = driver.findElement(By.id("errorMessage")).getText();
        assert errorMsg.contains("Invalid credentials");
        driver.quit();
    }
}

```

Download for reference  Like  Share 

Software and Testing Training YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

Software Testing Space Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

Question: How do you configure the test runner class for Cucumber?

Answer: The **TestRunner.java** file is used to run the Cucumber tests. You define the location of the feature files and step definitions and configure various options for the Cucumber framework. The example of **TestRunner.java** is:

```
import org.junit.runner.RunWith;
import io.cucumber.junit.Cucumber;
import io.cucumber.junit.CucumberOptions;

@RunWith(Cucumber.class)
@CucumberOptions(
    features = "src/test/resources/features",
    glue = {"stepDefinitions"},
    plugin = {"pretty", "html:target/cucumber-reports"},
    monochrome = true,
    tags = "@SmokeTest"
)
public class TestRunner {
    // This class will run the tests
}
```

Explanation of options:

- features: The path to the folder where your feature files are located.
- glue: The package where your step definitions are located.
- plugin: Specifies the type of report to be generated after the test run (e.g. HTML, JSON).
- monochrome: If set to true, it makes the console output more readable.
- tags: Used to specify which tests to run (e.g., only @SmokeTest tagged tests).

Question: What are the important components of a Cucumber BDD framework?

Download for reference  Like  Share 

[Software and Testing Training](https://youtube.com/@QA1) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](https://inderpsingh.blogspot.com/) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

Answer: The components of a Cucumber BDD framework are:

- **Feature Files:** These contain the test scenarios written in Gherkin. Each feature file describes a specific feature of the application under test.
- **Step Definitions:** These are Java methods that link the Gherkin steps to the code responsible for performing the actions. Each Gherkin step has a corresponding step definition.
- **Hooks:** These are setup or teardown methods (like `@Before`, `@After`) that execute before or after each scenario to prepare the test environment or clean up resources.
- **Test Runner:** This is a configuration file (usually a Java class) that runs the Cucumber tests. It specifies the location of feature files, step definitions, and the reporting configurations.
- **Configuration Files:** For Maven-based projects, configuration files such as `pom.xml` manage dependencies like Cucumber, JUnit, TestNG, or Selenium.
- **Reports:** Cucumber provides built-in support for generating test reports in various formats like HTML or JSON. These reports summarize the test results and can be integrated with CI tools like Jenkins.

Question: How can this framework be expanded for real-world applications?

Answer: This framework can be easily expanded to cover more complex scenarios and functionality:

- **Data-Driven Testing:** Use Cucumber's Scenario Outline and Examples table to run the same test with different sets of data.
- **Parallel Execution:** Integrate Cucumber with tools like **TestNG** or **Cucumber-JVM Parallel Plugin** to run tests in parallel, significantly reducing test execution time.
- **CI/CD Integration:** Add continuous integration by integrating the project with **Jenkins**, enabling automated builds and test execution on code changes.
- **Rest Assured for API Testing:** Extend the framework to handle API testing by integrating it with **Rest Assured**, allowing the automation of both UI and API tests under the same Cucumber framework.

Download for reference  Like  Share 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

12. Handling and Troubleshooting Cucumber BDD Issues

Question: What are some common errors in Cucumber, and how can they be troubleshooted?

Answer: Common errors in Cucumber relate to missing step definitions, misconfigured project setup, or syntax issues in the feature files. A few examples are:

- **Undefined Step Definitions:** This error occurs when the steps written in the feature file do not have corresponding step definitions. **Solution:** Check that the regular expressions in your step definitions match the steps in your feature file. You can also copy the unrecognized steps from the console output, which Cucumber suggests, and implement the corresponding Java method.
- **Cucumber-JVM Plugin Errors:** Errors related to the Cucumber Maven or Gradle plugins, usually caused by incorrect plugin versions or incompatible dependencies. **Solution:** Check the `pom.xml` (for Maven) or `build.gradle` (for Gradle) file to check that the correct versions of Cucumber, JUnit/TestNG, and other dependencies are specified. Running `mvn clean install` or `gradle clean build` can help identify and fix dependency issues.
- **Gherkin Syntax Issues:** These errors occur when there's a problem with the syntax in the feature file, such as misusing keywords like `Given`, `When`, `Then`. **Solution:** Check the Gherkin syntax for accuracy. Ensure that indentation and keyword usage are correct and that all scenarios are properly formatted.

Question: What can you do to resolve step definition recognition issues in Cucumber?

Answer: Step definition recognition issues happen when Cucumber cannot find the step definitions corresponding to the steps in the feature file. Here are some common causes and their solutions:

- **Incorrect Glue Path:** The `glue` parameter in the runner class specifies where the step definitions are located. If this is not configured correctly, Cucumber won't be able to locate the step definitions. **Solution:** Ensure that the `glue` attribute in the

Download for reference  Like  Share 

[Software and Testing Training](https://www.youtube.com/@QA1SoftwareandTestingTraining) YouTube Channel 81.3K Subscribers [https://youtube.com/@QA1](https://www.youtube.com/@QA1SoftwareandTestingTraining)

[Software Testing Space](https://inderpsingh.blogspot.com/) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

@CucumberOptions annotation points to the correct package where your step definition classes are located: @CucumberOptions(glue = {"stepDefinitions"})

- **Mismatch Between Feature Steps and Step Definitions:** If there is a typo or mismatch between the feature file step and the step definition, Cucumber won't recognize it. **Solution:** Verify that the step text in the feature file exactly matches the regular expression in the step definition. Check for spaces, punctuation, and case sensitivity.
- **Missing Step Definition Annotations:** If a step definition method is missing the proper annotation (@Given, @When, @Then), it won't be recognized by Cucumber. **Solution:** Make sure all step methods are correctly annotated and contain valid regular expressions. For example: @Given("^the user is on the login page\$")

Question: What are common issues encountered during parallel execution in Cucumber, and how can they be resolved?

Answer: Parallel execution issues in Cucumber often arise from improper resource handling, concurrency problems, or limitations with certain testing frameworks. Here're a few solutions to tackle such issues:

- **Shared State Between Scenarios:** If scenarios share state (e.g. variables, browser instances), this can cause interference between parallel tests. **Solution:** Use the @Before and @After hooks to initialize and clean up resources separately for each scenario. Avoid using static variables or shared instances between tests. Cucumber also offers a Scenario object that provides methods to maintain scenario-specific state.
- **Thread-Safety in Step Definitions:** When executing tests in parallel, it's important that step definitions and any related resources are thread-safe. **Solution:** Confirm that any global variables or objects used in the step definitions are handled in a thread-safe manner, either by using local variables or synchronized methods where necessary.
- **Framework Configuration:** Some frameworks, like JUnit, may have limited parallel execution support compared to TestNG. **Solution:** If you're using JUnit, consider switching to TestNG for more robust parallel execution capabilities. If you must use JUnit, confirm that the cucumber-jvm-parallel-plugin is properly configured for parallel execution.
- **Dependency Injection Issues:** If you use dependency injection frameworks like Spring or Guice, improper configuration might lead to concurrency issues. **Solution:** Make sure the DI container is configured to provide scenario-scoped instances (a fresh instance for each test execution).

Download for reference  Like  Share 

[Software and Testing Training](https://www.youtube.com/@QA1SoftwareandTestingTraining) YouTube Channel 81.3K Subscribers [https://youtube.com/@QA1](https://www.youtube.com/@QA1SoftwareandTestingTraining)

[Software Testing Space](https://inderpsingh.blogspot.com/) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

Question: What can you do if Cucumber test reports are missing or incomplete?

Answer: Issues with Cucumber reports are generally caused by misconfiguration in the runner class or problems with the reporting plugins.

- **Missing Reports:** If Cucumber reports are not being generated, check the plugin option in the test runner. The correct plugin should be specified in `@CucumberOptions`. For HTML reports:
`@CucumberOptions(plugin = {"html:target/cucumber-reports"})`
- **Incomplete or Blank Reports:** This could be due to parallel execution or a plugin misconfiguration. **Solution:** Check that all steps have a valid status in your feature files (i.e. no steps left unimplemented). When using parallel execution, confirm that the reports from each thread are correctly aggregated.

Question: What strategies do you apply for handling flaky tests in Cucumber?

Answer: Flaky tests are a common problem in test automation, often due to timing issues, inconsistent environments, or race conditions. Here are some strategies to handle them in Cucumber:

- **Use Explicit Waits:** Avoid using thread sleeps or hardcoded waits. Instead, use explicit waits to wait for a certain condition to be met (like waiting for an element to be visible).
- **Environment Stability:** Make sure that the environment where tests are executed is stable. Test failures due to network issues, server crashes, or inconsistent environments should be minimized.
- **Retries for Flaky Tests:** Some frameworks like TestNG allow for retry mechanisms to automatically rerun failed tests. Integrating retries can help reduce the impact of flaky tests.
- **Isolate Test Cases:** Create test cases such that each test case is independent and does not rely on the outcome of previous test cases. This can prevent false negatives caused by earlier tests.

If you found my document useful, I would appreciate if you check out my [Software and Testing Training](#) channel and [subscribe](#) to it. Thank you!

Download for reference  Like  Share 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

Download for reference  **Like**  **Share** 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

13. CI/CD Integration with Cucumber

Question: How can you integrate Cucumber BDD into a CI/CD pipeline using Jenkins or other tools?

Answer: Integrating Cucumber BDD into a CI/CD pipeline makes automated testing a part of every build process, providing continuous feedback on code quality. Here's a step-by-step guide to integrating Cucumber with Jenkins:

- **Set Up Jenkins:** First, ensure Jenkins is installed and configured. If using other CI tools like GitLab CI, Travis CI, or CircleCI, the steps will be similar.
- **Configure Jenkins Job:** Create a new Jenkins job (Freestyle or Pipeline). Set up the source code repository (Git, SVN) to pull the Cucumber BDD project automatically. If using a pipeline, define the pipeline script in the Jenkinsfile.
- **Install Necessary Plugins:** Jenkins requires plugins for reporting and running tests, such as JUnit Plugin (for test result tracking) and Cucumber Reports Plugin (for Cucumber HTML reports).
- **Configure Build Steps:** In a Freestyle job, add build steps to compile the project and run tests using Maven or Gradle. If using a pipeline, add a build stage to run Cucumber tests via commands like `mvn clean test`.
- **Publish Cucumber Reports:** In the post-build actions, use the Cucumber Reports Plugin to publish the generated reports. Specify the path where the Cucumber test reports (HTML/JSON) are stored: `target/cucumber-reports/*.json`.
- **Trigger Builds:** Optionally, configure triggers (like webhooks) to run Cucumber tests after every commit or merge in the repository, ensuring tests run automatically in every CI pipeline.

Question: How can you run automated Cucumber tests in CI pipelines?

Answer: Running automated Cucumber tests in a CI pipeline, such as Jenkins, GitLab CI, or Travis CI, involves configuring the build pipeline to execute Cucumber tests as part of the build and deployment process. Here's how it can be done:

Download for reference  Like  Share 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

- **Pipeline Configuration:** Create a Jenkinsfile (for Jenkins) or .gitlab-ci.yml (for GitLab CI) to define the stages of the CI pipeline. Include a testing stage where Cucumber tests are executed. **Example Jenkins Pipeline Script:**

```
pipeline {  
    agent any  
  
    stages {  
        stage('Build') {  
            steps {  
                sh 'mvn clean install'  
            }  
        }  
  
        stage('Test') {  
            steps {  
                sh 'mvn test'  
            }  
        }  
  
        stage('Report') {  
            steps {  
                publishCucumberReports(fileIncludePattern:  
'target/cucumber-reports/*.json')  
            }  
        }  
    }  
}
```

Download for reference  Like  Share 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

}

- **Execution:** When a commit or merge occurs in the code repository, the pipeline automatically triggers, running the Cucumber tests in the test stage. The build will fail if any of the tests fail, alerting the team to issues in the new code.
- **Parallel Execution:** To speed up test execution, you can configure your CI pipeline to run Cucumber tests in parallel. This can be achieved using tools like TestNG or JUnit with the Cucumber-JVM plugin.

Question: How can you monitor and report Cucumber test results in CI?

Answer: Monitoring and reporting Cucumber test results in a CI/CD pipeline can be done using Jenkins and its reporting plugins. Here's how:

- **Generating Reports:** When the Cucumber tests run, they generate reports in JSON or HTML format. These reports detail the test execution, including passed and failed scenarios, step execution times, and error logs.
 - **HTML Report:** A simple, readable report that provides a high-level view of the test execution.
 - **JSON Report:** A structured format that CI tools can parse for deeper analysis and insights.
- **Cucumber Reports Plugin:** Use the Jenkins Cucumber Reports Plugin to parse JSON reports and generate visual dashboards showing the status of the test scenarios.
Configuration Example: In Jenkins, add the post-build action Publish Cucumber Reports. Set the report file location such as, target/cucumber-reports/*.json
- **JUnit Reporting:** Cucumber also integrates with JUnit reporting. After running the tests, JUnit-compatible XML files are generated, which Jenkins can display in the Test Result section.
- **Email Notifications:** Jenkins can be configured to send automated email notifications when test failures occur. You can set conditions to trigger emails only when the build fails or if tests are unstable, ensuring timely updates to the team. **Example Jenkins Email Notification:**
Use the Email Extension Plugin to configure email triggers after a build finishes:

Download for reference  Like  Share 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.

```
emailx subject: "Build ${BUILD_STATUS}: Job '${JOB_NAME} [${BUILD_NUMBER}]'",  
  body: "${BUILD_URL} - Test Results",  
  to: 'team@example.com',  
  attachLog: true
```

Download for reference  Like  Share 

[Software and Testing Training](#) YouTube Channel 81.3K Subscribers <https://youtube.com/@QA1>

[Software Testing Space](#) Blog <https://inderpsingh.blogspot.com/>

Copyright © 2025 All Rights Reserved.